AI Clinic Voice Receptionist – Build & Deployment Guide

Version 2.1 · 11 November 2026 · Updated from version 1 (7 November 2026) with what was actually built, how it was tested, what was measured, and what changes for production. Version 2.1 adds the short intake-only call flow the client asked for.

Client location: Calgary, Alberta, Canada. **Development and testing:** India. **Production deployment:** Canada (AWS `ca-west-1`, Calgary).

Model: one dedicated agent per clinic (single-tenant). Shared core codebase, separate config and deployment per clinic.

All prices are approximate and in USD. Vendor prices and free tiers change often, so check each vendor's pricing page before you quote a client.

What changed since version 1

- **New in 2.1: the client asked for a simpler call.** The agent now only takes the caller's request, name and number and promises a callback from the front desk within 30 minutes (§1). The full receptionist (booking, cancelling, messages, transfers, FAQs) is still in the code, switched off by commenting it out, and can be turned back on.
- **Phase 0 and Phase 1 are built and tested.** The whole system (agent, backend, database, staff dashboard) runs on one laptop and has handled real browser calls through LiveKit, including a call by the developer in the LiveKit Playground on 11 Nov 2026.
- **Development runs on local, free components:** Qwen3 8B on **Ollama** for the LLM and self-hosted **Kokoro** for the voice. No patient data and no test audio leave the laptop for those two legs. Cartesia stays the production voice; switching is one line in `clinic.yaml`.
- **Timed tests use Groq** (Qwen 3.8 27B). The best measured run: **1.21 s** median from the end of the caller's sentence to the first sound of the reply, under the 1.5 s target.
- **Safety rules are enforced in code**, not only in the prompt: emergency (911) and crisis (988) detection before the LLM, a medical-advice filter on every reply, and a read-back check before any booking.
- **Correction:** v1 gave the latency target as "about 1 second" in one place. The target used throughout is **under 1.5 s** median per reply.
- New sections: how it was tested (§6), latency results and tuning (§7), and **what changes for production** (§8).

1. What the system does

> ### Current call flow: intake only (client request, 11 Nov 2026)
>
> 1. The AI answers and introduces itself: AI disclosure, recording notice and the 911 line, then *"Do you have a health question, or would you like to schedule an appointment?"*
> 2. For an appointment it asks which service; for a health question it asks what it is about (and never answers it); anything else is noted in a few words.
> 3. It asks for the caller's full name and best callback number, and reads the number back as single digits for the caller to confirm.
> 4. It saves the request and says the clinic's fixed closing line: *"Our front desk receptionists are busy with other patients right now, but someone will call you back within 30 minutes. Goodbye."* Then it ends the call.
>
> The dashboard lists each call with **caller name, phone number, reason for the call and summary** (plus the time of the call, for the 30-minute promise). Emergency (911) and crisis (988) detection, the medical-advice filter, silence handling and the LLM-failure fallback all stay on.
>
> The greeting and the closing line are in `clinic.yaml` (`greeting`, `callback_message`), so each clinic can word them. The agent has one tool in this flow, `save_callback_request`.

Full receptionist (built and tested, currently switched off). A patient calls the clinic's Calgary phone number. The AI answers, understands what they say, replies in a natural voice and can:

- Answer FAQs (hours, location, services, parking, insurance), using only what is in the clinic's config file
- Take booking, rescheduling and cancellation requests (staff confirm them in the dashboard until the clinic's booking system is connected)
- Take messages for the front desk
- Transfer urgent or clinical calls to a human, or tell callers to hang up and dial 911 in an emergency (988 for a mental-health crisis)

Every call is saved with its transcript, a short summary and any request, and appears on the staff dashboard within about a second of hang-up.

Switching back is uncommenting: the five tools and the full prompt in `core/agent.py`, the old greeting in `clinic.yaml`, the old guardrail wording in `core/guardrails.py` and the old dashboard columns. The steps are written next to the commented code. The logic behind the five tools was never commented out and is still covered by the automated tests.

2. Technology stack: development vs production

Layer	Development and testing (built, in use)	Production	Notes
Phone number / carrier	None yet; browser calls via the LiveKit Playground	Twilio Calgary number (403/587/825)	Phase 2. Telnyx is an alternative.

Call bridge	—	Twilio Elastic SIP trunk → LiveKit inbound trunk + dispatch rule	Phase 2
Real-time media	LiveKit Cloud (free tier)	LiveKit Cloud with a confirmed data region, or self-hosted LiveKit in Canada	Recording off (`record=False`)
Agent framework	LiveKit Agents (Python), version 1.8; runs in a Docker container		Docker because Windows Smart App Control blocks LiveKit's native library on the dev laptop
Speech-to-text	Deepgram Nova-3, streaming		Production needs a DPA and zero retention (§9)
LLM	**Ollama,** `qwen3-clinic` (Qwen3 8B) for everyday work; **Groq** `qwen/qwen3.8-27b` for timed tests	Paid hosted API with a DPA, or Qwen 7B–14B on vLLM in `ca-west-1`	Any OpenAI-compatible endpoint; switching is a `.env` change
Text-to-speech	**Kokoro** (self-hosted Kokoro-FastAPI, GPU or CPU container)	**Cartesia** (default), or Kokoro on a Canadian GPU server for a fully Canadian stack	Switching is `voice.tts_provider` in `clinic.yaml`
Voice activity detection	Silero VAD (built into LiveKit)		End-of-turn silence is a per-clinic setting (§7)
Backend / tools	FastAPI (Python)		The agent never touches the database or booking system directly
Database	PostgreSQL 16 in Docker (SQLite also works without Docker)	AWS RDS PostgreSQL in `ca-west-1`, encrypted at rest	Alembic migrations
Staff dashboard	Next.js on `localhost:3000`	Same app behind HTTPS on the Canadian server	
Booking	Manual adapter: request recorded, staff confirm in the dashboard	The clinic's system (Jane App, Google Calendar, Accuro, Oscar, Cliniko) once named	Phase 3; manual keeps working meanwhile
Staff notifications	Printed to the backend log	SendGrid / SMTP email or Twilio SMS, with no patient details in the text	`NOTIFY_CHANNEL`
Secrets	Per-clinic `.env`, git-ignored	AWS Secrets Manager	Never in the repo or image
Hosting	One laptop (RTX 5050 8 GB, Docker Desktop)	AWS `ca-west-1` (Calgary)	Phase 5

Keep the LLM at 7B–14B parameters for speed. One exception for testing: Groq currently serves no Qwen model in that range, so timed tests used its 27B Qwen, which was still fast (0.29 s to first token).

3. Architecture: how each part talks to the next

Call flow, step by step (unchanged from v1, with the latency target corrected):

1. **Patient → Twilio:** the patient dials the clinic's Calgary number (PSTN).
2. **Twilio → LiveKit:** Twilio forwards the call over SIP. LiveKit creates a room and adds the caller.
3. **LiveKit → agent:** LiveKit dispatches the Python agent into the room. Audio streams both ways over WebRTC.
4. **Agent → Deepgram:** caller audio goes over a WebSocket; live text comes back.
5. **Agent → guardrails → LLM:** every caller message is screened for emergency and crisis language first. A match plays a fixed 911 or 988 message and never reaches the LLM. Otherwise the text, the conversation history and the clinic's system prompt go to the LLM (OpenAI-compatible API, streaming).
6. **LLM → tools:** in the intake flow, one tool saves the caller's request; in the full receptionist, the LLM checks slots, books, cancels, takes messages or transfers. Either way the LLM emits a tool call. The agent validates it and calls the FastAPI backend, which talks to the booking system and database.
7. **LLM → advice filter → TTS:** each sentence of the reply is checked for medical advice before it is spoken, then streamed to the TTS.
8. **Agent → LiveKit → Twilio → patient:** audio plays to the caller. Target: **under 1.5 s median** from the end of the caller's sentence to the first sound of the reply.
9. **After the call:** transcript, summary and any request are saved to PostgreSQL and pushed to the dashboard; urgent calls trigger a staff alert.

```
Patient phone → Twilio (PSTN) → SIP trunk → LiveKit room → Python agent → [Deepgram STT →
guardrails → LLM (Qwen) → advice filter → TTS] → LiveKit → Twilio → patient
Side channel: LLM tool call → FastAPI backend → booking system / email / SMS / PostgreSQL →
dashboard (WebSocket)
```

Two paths, kept separate: the voice loop never waits on the database. Call records are written in the background; tool calls are the only backend requests made while the caller waits, and they have short timeouts. If the backend is down, messages are kept on disk and replayed later.

From	To	Protocol
Phone network	Twilio	PSTN
Twilio	LiveKit	SIP (Elastic SIP Trunk)
LiveKit	Agent	WebRTC / LiveKit SDK
Agent	Deepgram	WebSocket streaming
Agent	LLM	HTTPS, OpenAI-compatible chat completions (streaming)
Agent	TTS	Cartesia: WebSocket streaming. Kokoro: HTTPS, OpenAI-compatible speech API (raw 24 kHz audio)
Agent	Backend	HTTPS REST (JSON), shared-secret token

Backend	Booking system	REST API or webhook
Backend	Dashboard	HTTPS REST + WebSocket push; session cookies
Backend	Clinic staff	Email (SMTP/SendGrid) or SMS (Twilio)

---

4. Single-tenant setup (one agent per clinic)

- One core Git repository. Each clinic is a folder: `clinics/<clinic>/clinic.yaml` (name, hours, services, FAQs, transfer number, voice, greeting, limits; committed) and `clinics/<clinic>/.env` (keys; never committed).
- Nothing clinic-specific is written in the code. Adding a clinic means copying the folder and setting `CLINIC=<folder>`; a test proves a second clinic runs with no code change.
- `clinic.yaml` is checked when the agent starts, not mid-call: it won't load unless the greeting mentions AI and recording.
- Each clinic gets its own deployment: separate container set, Twilio number, LiveKit dispatch rule and database. Bug fixes go into the core repo once, then each clinic is redeployed.

```
core/        agent.py, tools.py, prompts.py, guardrails.py, config.py, settings.py, providers/
(stt, llm, tts)
backend/     FastAPI app, booking adapters, auth, audit log, WebSocket, Alembic migrations
dashboard/   Next.js staff dashboard
clinics/     demo_clinic/clinic.yaml (+ .env, not committed)
deploy/      Dockerfiles, docker-compose.yml, docker-compose.kokoro-gpu.yml
tests/       pytest suite + scenarios/ (text and voice test callers)
```

5. Build procedure: how it was actually done

Phase	What it covers	Status (11 Nov 2026)
0 – Accounts and prerequisites	Vendor keys, local tooling, Docker stack, key check at startup	**Done (Twilio keys wait for Phase 2)**
1 – Full local vertical slice	Config, database, backend, booking adapter, agent, tools, guardrails, dashboard, testing	**Done: all 5 exit criteria met (§6)**
2 – Telephony	Twilio number, SIP trunk, real phone calls	Next
3 – Booking integration	The clinic's booking system	**Waiting on the clinic to name it**
4 – Hardening, privacy, compliance	Fallbacks, encryption, retention, DPAs, PIA	Partly done
5 – Canadian deployment, soft launch	AWS `ca-west-1`, staged rollout	Not started

Step 1 – Accounts

LiveKit Cloud (free tier), Deepgram (free credit), Cartesia (free tier; its free credit ran out during testing), Groq (free tier, no card), OpenRouter (optional; its free tier is capped at 50 requests a day until $10 of credit is bought). Twilio is only needed from Phase 2. Every key is checked at startup and with `python -m core.check_env --live`, which calls each vendor to prove the key works.

Step 2 – Local stack

- Python 3.11+, Node.js 20+, Docker Desktop, Git, Ollama.
- `docker compose -f deploy/docker-compose.yml --profile agent up -d --build` starts Postgres, backend, dashboard, the voice agent and Kokoro. The GPU override file runs Kokoro on the NVIDIA GPU.
- The agent runs in Docker, not natively, because Windows Smart App Control blocks LiveKit's native library. Do not switch Smart App Control off for this.
- Tested in the browser with the LiveKit Agents Playground: no phone number needed, free.

Step 3 – The LLM (Qwen)

- **Local:** `ollama pull qwen3:8b`, then `ollama create qwen3-clinic -f Modelfile.qwen3`. The Modelfile sets an 8k context (Ollama's default 2k cuts off the system prompt) and a low temperature.
- **Lessons from testing:**
  - Ollama ignores Qwen3's `/no_think`. Send `{"reasoning_effort": "none"}` or every reply "thinks" for 2–6 s.
  - On Windows use `127.0.0.1`, not `localhost`: localhost tries IPv6 first and added about 2 s per request.

  - Qwen3 8B passed 22/22 text scenarios and 8/8 multi-step runs; Qwen 2.5 7B finished multi-step flows only 4 times in 8. Qwen3 8B is the local default.
  - The agent has the LLM read the system prompt while the greeting plays (a "warm-up"), cutting the first reply's LLM time from 1.6 s to 0.12–0.5 s on Ollama. Turned off on Groq, where it only uses up the rate limit.
  - A stalled LLM request is abandoned after 4 s and retried at once. If the LLM fails for good, the caller hears an apology and the clinic's number, never silence.
- **Groq:** same settings with Groq's URL, key and `qwen/qwen3.8-27b`. Fast, but the free tier allows only about 7,000 input tokens per minute (see §11).

Step 4 – Tools and guardrails

Five tools, each calling the FastAPI backend: `check_availability`, `book_appointment`, `cancel_appointment`, `take_message`, `transfer_to_human`. Built-in protections:

- Every tool payload is validated; on the first error the LLM gets a correction, on the second it is told to take a message instead.
- **Read-back in code:** booking, cancelling and messages are refused until the agent has read the phone number (and date and time) back and the caller has said yes. Small models get digits wrong; this stops a wrong number being saved.
- While a tool runs, the agent says a short line ("Let me check that for you") if it hasn't already said something, so the caller doesn't sit in silence.
- If the agent promises a callback but nothing was saved, the call is flagged "Needs action" on the dashboard.
- **Intake flow:** only `save_callback_request` (name, number, appointment / health question / other, details) is active. It uses the same read-back check on the phone number, then plays the fixed closing line and ends the call, so the model can't promise anything else.
- **Found with the real 8B model and fixed in code:** it sometimes repeated the greeting in every reply, and once read a safety rule from its own instructions aloud. Both are now removed from replies before they are spoken (the 9-1-1 sentence is never removed).
- **Found on a live call and fixed:** a number spoken as "seven zero, double two, ..." was counted one digit short, so the read-back check never matched and the agent kept asking. "Double" and "triple" are now understood, Deepgram returns numbers as digits, the agent reads numbers back as single digits, and after two failed checks a number the caller confirmed is saved with a "check the number" note instead of looping.
- Limits from `clinic.yaml`: re-prompt after silence, then hang up; maximum turns; maximum call length.

Step 5 – Booking

The manual adapter ships first: availability comes from the clinic's hours, the request is recorded and staff confirm it from the dashboard. The clinic's own system (Jane App, Google Calendar, Accuro, Oscar or Cliniko) plugs in behind the same interface once they name it. If it's Jane App, apply for API access early: it is limited.

Step 6 – Voice (TTS)

Development uses self-hosted **Kokoro** (Kokoro-FastAPI, voice `af_heart`), so testing costs nothing and no text leaves the machine. On the RTX 5050 (a Blackwell GPU) the upstream GPU image fails ("no kernel image is available"); `deploy/Dockerfile.kokoro-gpu` swaps in a CUDA 12.8 PyTorch build. Production uses **Cartesia** by changing `tts_provider: cartesia` and the voice ID in `clinic.yaml`; the Cartesia code path is kept live and unchanged.

Step 7 – Real phone number (Phase 2, next)

1. Buy a Calgary number in Twilio (403, 587 or 825).
2. Create an Elastic SIP trunk; set its origination URI to the LiveKit SIP URI; attach the number.
3. In LiveKit, create an inbound SIP trunk and a dispatch rule that puts callers in a room with the agent.
4. The agent already reads the caller's number from the SIP `From` header; verify it on real calls.
5. Test from India with a SIP softphone (Zoiper) or Twilio Voice in the browser to avoid international charges.

Step 8 – Deploy to Canada (Phase 5)

See §8 for the full list of production changes.

---

6. Testing: how it was tested and the results

All testing uses **fake patient data only**: fictional names and 403-555-01xx numbers.

Layer	What it does	Result
Automated tests (`pytest`)	Guardrails, tools, backend, audit log, migrations, the second-clinic check, and the real agent inside a LiveKit session driven by a scripted LLM. No keys needed.	**233 pass (2 skipped: tests of a full-receptionist tool that is switched off)**
Text scenarios against the real LLM	22 scripted conversations (FAQ, booking, cancel, message, transfer, emergency, advice bait, wrong number…), a second LLM playing the caller, checks done in code	**22/22 (Qwen3 8B)**
Intake flow, text scenarios against the real LLM	Appointment, health question and "something else" requests, plus 10 emergencies, crisis, 4 advice-bait questions and a wrong number	**19/19 (Qwen3 8B on Ollama)**
Voice calls with a synthetic caller	A test caller joins a LiveKit room and speaks; 7 scenarios: greeting talked over, FAQ, background noise, interrupting, emergency, advice bait, booking, silence. Measures latency the way a caller hears it.	**7/7 on local models; 5/7 on Groq (2 failed on Groq's rate limit)**
Human call in the LiveKit Playground	The developer called with a real microphone and ran the scenario list	**Working (11 Nov 2026)**
Human call, intake flow	The developer called through the new flow in the Playground	**Working after the spoken-number fix (§5, Step 4)**

Phase 1 exit criteria

Criterion	Evidence
A browser call books an appointment; it appears in Postgres and on the dashboard within a second of hang-up	**Met: booking calls over LiveKit; request on the dashboard 0.05 s after hang-up**
Emergency phrasing produces the 911 response every time, across 10 attempts	**Met: 10/10 in code tests, 10/10 with the real LLM, and by voice (911 message in 0.60 s)**
No medical advice in any scripted scenario	**Met: none in 22 text scenarios (4 of them advice bait) or in voice runs**
Median reply latency under 1.5 s, measured and written down	**Met on the Groq + GPU Kokoro run: 1.21 s as the caller hears it, 1.08 s measured by the agent.**
A second clinic folder runs with no code change	**Met: automated test**

Caveat on latency: the Groq free tier rate-limited longer calls, so the 1.21 s median comes from shorter calls. Re-measure on the production LLM and on real phone calls (Phase 2).

---

7. Latency: what was measured and what was tuned

Setup (all on the dev laptop, LiveKit Cloud)	Caller hears (median)	Agent measures (median)	Main limit
OpenRouter Qwen 9B + Cartesia (9 Nov)	~1.9 s	~1.5 s	End-of-speech wait, LLM host
Ollama Qwen 2.5 7B + Cartesia, warm-up on	2.01 s	1.39 s	Tool-call turns
Ollama Qwen3 8B + Cartesia	1.85 s	1.42 s	(Cartesia credit ran out)
Ollama Qwen3 8B + Kokoro on CPU	—	2.71 s	Kokoro on CPU: 0.7–4.2 s to first audio
Ollama Qwen3 8B + Kokoro on GPU (shared)	2.05 s	2.03 s	LLM and TTS sharing one 8 GB GPU
Groq Qwen 3.8 27B + Kokoro on GPU (11 Nov)	**1.21 s** (p90 2.18 s)	**1.08 s**	Groq free-tier rate limit

Where the time goes on the best run (per reply): end-of-speech wait 0.42–0.48 s, speech-to-text 0.24–0.47 s, LLM first token 0.38–0.45 s, Kokoro first audio 0.10–0.19 s (Cartesia measured 0.13–0.17 s).

What was tuned:

- **End-of-turn silence** (`voice.end_of_turn_silence` in `clinic.yaml`): how long the caller must be silent before the agent answers. Silero's built-in 0.55 s was the real floor, so lowering LiveKit's delay alone did nothing; both are now set from this one value. The demo clinic uses 0.4 s. Lower is faster but can cut in when a caller pauses while reading out a number; re-tune on real phone calls.

- Prompt warm-up during the greeting (first reply 1.6 s → 0.12–0.5 s on Ollama).
- Qwen3 "thinking" off; `127.0.0.1` instead of `localhost` on Windows.
- A spoken hold-on line during tool calls (booking turns took 4–6 s on local models).
- LLM request timeout 4 s with an immediate retry (a real call once stalled 19 s).

8. What changes for production

Everything built so far runs unchanged in production. These are the settings, services and paperwork that change, and the phase that delivers each.

Area	Now (development)	Production	Phase
Phone	Browser calls only (Playground)	Twilio Calgary number → Elastic SIP trunk → LiveKit inbound trunk + dispatch rule. The clinic's own number connects by forwarding, porting or direct SIP (§13).	2
Human transfer	Simulated (no phone line on a browser call)	Real SIP transfer to the clinic's urgent line, tested end to end	2
LLM	Ollama (local) and Groq free tier	A **paid** hosted API with a DPA and zero data retention, or Qwen 7B–14B on vLLM in `ca-west-1`. Free tiers can't serve real calls: Groq's free tier cut calls off after about two turns.	4–5
Model choice	Qwen3 8B (local), Qwen 3.8 27B (Groq)	Decide by measurement on the production provider: 7B–14B as briefed, unless a larger model is as fast there	4
Voice (TTS)	Kokoro, self-hosted on the laptop	Cartesia (`tts_provider: cartesia`) with a DPA and zero retention, or Kokoro on a Canadian GPU server if the clinic wants no US voice vendor	4
Speech-to-text	Deepgram (free credit)	Deepgram paid plan with DPA and zero retention, or faster-whisper in Canada	4
Never dead air	LLM failure → spoken apology + clinic number	Any failure (LLM, **TTS**, agent down) → the call forwards to the clinic line or voicemail. A TTS failure is still dead air today: must fix.	4
Hosting	Docker Compose on one laptop	AWS `ca-west-1`: ECS or small EC2 for agent + backend; dashboard behind HTTPS (ALB/CloudFront); one task set per clinic	5
Database	Postgres in a container, unencrypted	RDS PostgreSQL, encrypted at rest, nightly encrypted backups (restore tested once); transcripts auto-deleted after an agreed window	4–5

LiveKit	LiveKit Cloud, default region	Confirmed Canadian/acceptable data region, or self-hosted LiveKit in `ca-west-1`; recording stays off	5
Secrets	`.env` file per clinic	AWS Secrets Manager; nothing on disk or in images	5
Staff alerts	Printed to the log	Email (SendGrid/SMTP) or SMS (Twilio), with no patient details in the text	4
Booking	Manual (staff confirm)	The clinic's booking system, or stays manual	3
Security	Login with session cookies; audit log	Plus backend rate limiting, HTTPS everywhere, optional two-factor sign-in, session timeout reviewed	4
Monitoring	Container logs	CloudWatch dashboards and alerts: call failures, latency over 2 s, provider errors, agent worker down, with a named person on the other end	5
Settings	`ENVIRONMENT=development`	`ENVIRONMENT=production`; real clinic content in `clinic.yaml` in the clinic's own words	5
Call flow	Intake only: request, name, number, "the front desk will call you back within 30 minutes"	The clinic must be able to keep the 30-minute promise whenever the AI answers. Add an after-hours closing line (agreed, to do) so callers aren't promised 30 minutes at night or on closed days. Turn the full receptionist back on if the clinic later wants booking by phone.	2–4
Rollout	—	After-hours only → overflow (after 4 rings) → all calls, a week each with transcript review; rollback plan to turn the AI off in under five minutes	5

Must be re-done or re-measured in production

- **Latency** on real phone calls with the production LLM and Cartesia: the phone network adds delay that browser tests don't show. Target still under 1.5 s median.
- **End-of-turn silence** on real phone audio and real callers, especially older callers and anyone reading out a number.
- **Emergency and no-advice checks** by voice after every prompt change.
- **Interrupting**: stop time measured 0.5–2.2 s across runs (limit 1.5 s); recheck on the phone path.

Known gaps today

- TTS failure leaves the caller in silence (Phase 4 fallback above).
- The closing line promises a callback within 30 minutes at any hour, including nights and closed days. An after-hours wording is agreed and still to be built.
- The 911 message says "stay on the line and I'll connect you", but call transfer is switched off in the intake flow. Left as is by decision.
- (Full receptionist) The answer to "what services do you offer" is weak; the prompt should list services from `clinic.yaml`.
- The hold-on line and the 0.4 s end-of-turn setting are covered by automated tests but have little live-call history yet.

---

9. Privacy and compliance (Alberta)

- **Alberta Health Information Act (HIA)** applies to health information handled by custodians such as clinics. The clinic is the custodian; you are their affiliate or service provider. PIPEDA / PIPA (Alberta) may also apply.
- **During development:** fake patient data only. With Ollama and Kokoro the LLM and voice legs stay on the laptop; Deepgram, LiveKit and Groq still process test audio or text in the US.
- **Before go-live (non-negotiable):** cloud voice APIs send patient audio and transcripts outside Canada, so the clinic must accept that **explicitly and in writing**, with DPAs signed and the clinic's Privacy Impact Assessment signed off by their privacy officer.

Vendor in the call path	What it sees	Where
Twilio	Call audio, caller's number	US vendor; confirm processing region
LiveKit Cloud	Call audio in transit (recording off)	Depends on region; confirm
Deepgram	Caller audio, transcripts	US
LLM provider (hosted)	Transcripts, clinic prompt	US for Groq/OpenRouter/Together/DeepInfra; Canada if self-hosted
Cartesia	Reply text	US; Canada if replaced by self-hosted Kokoro
Email/SMS provider	Alert text (no patient details)	US

The AI disclosure and recording notice are in the greeting, which can't be interrupted, and the config won't load without them. The agent never gives medical advice: emergencies go to 911, clinical questions go to staff. Sign a written agreement with the clinic covering data handling. This is not legal advice: have the clinic confirm compliance with their privacy officer or legal adviser.

10. Costs

Testing phase (India): what it actually took

Item	Planned (v1)	Actual so far
LiveKit Cloud	Free tier	Free tier
Deepgram	~$200 free credit	Free credit
Cartesia	Free tier	Free credit ran out mid-testing; replaced by Kokoro (free, self-hosted) for testing
LLM	Free tier or a few cents	OpenRouter needed paid credit (HTTP 402 at $0); testing moved to Ollama (free, local) and Groq's free tier
Twilio	Trial credit (~$15)	Not used yet
Total	~$0–20	**Free tiers and local models only** (no paid credit recorded). Budget ~$10–20 for paid LLM and Cartesia credit for the final timed runs and Phase 2.

Production (per clinic, per month, approximately; unchanged from v1)

Item	Approximate price	For 1,000 call minutes/month
Twilio Calgary number	$1–2/month	$1–2
Twilio inbound voice / SIP	about $0.005–0.01/min	$5–10
LiveKit Cloud (or self-host)	about $0.01/min, or the server cost	$10
Deepgram STT	about $0.004–0.008/min	$4–8
Cartesia / ElevenLabs TTS	about $0.02–0.08/min	$20–80
LLM, hosted Qwen (paid tier)	fractions of a cent per call	$1–5
LLM, self-hosted Qwen on a GPU server	$300–800/month per GPU server	$300–800
Agent + backend server (AWS)	$20–60/month	$20–60
Database (RDS small)	$15–30/month	$15–30

Rough total per clinic: about $80–200/month with hosted APIs; about $400–1,000/month with a self-hosted GPU LLM. One GPU server can be shared across several clinics' deployments, with separate databases. A GPU server could also run Kokoro, cutting the TTS line to near zero; that only pays off across several clinics.

What to charge the client (suggestion): a one-time setup fee for the build, integration and testing, plus a monthly fee covering usage, maintenance and support, priced above running costs.

11. Hosted LLM APIs: what we learned

Provider	Models	Approx. price	Notes from testing (Nov 2026)
Groq	Open models; Qwen 3.8 27B, GPT-OSS 20B/120B on this account	about $0.10–0.80 per million tokens	Fastest tested: 0.29 s to first token. No 7B–14B Qwen offered. Free tier ~7,000 input tokens/min: a call resends ~2,500 tokens per request, so calls fail after about two turns. Paid tier needed. US.
OpenRouter	Qwen, GLM, many others (routes to other hosts)	depends on the host, often under $1 per million tokens	Qwen 9B: ~0.6 s to first token; hosts vary (pin the fast ones with tool support). Free tier: 50 requests/day until $10 credit. Location varies by host.
Together AI	Qwen, GLM, Llama	about $0.20–1.20 per million tokens	Among the fastest OpenRouter hosts measured. US.
DeepInfra	Qwen, GLM	about $0.05–0.80 per million tokens	Among the fastest OpenRouter hosts measured. US.
Alibaba Model Studio	Qwen (official)	low, varies by model	Not tested. Outside Canada.
Ollama (local)	Qwen3 8B, Qwen 2.5 7B	Free	Development default. Needs ~6 GB of GPU memory and enough RAM; stays on the machine.

Connecting any of them is a `.env` change: `LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL` and, if needed, `LLM_EXTRA_BODY`. **Privacy note:** the hosted ones process data outside Canada. Use them with fake data for testing; for production, self-host in Canada or get the clinic's written privacy sign-off.

12. Database and staff dashboard

Tables built (PostgreSQL, Alembic migrations): `calls` (caller, times, reason, urgency, outcome, summary, transcript, per-reply latency, status), `appointment_requests`, `messages`, `staff_users`, `audit_log` (every staff read and write of a patient record). Without it, messages vanish when the call ends and there is no record for staff or audits. In production: RDS in `ca-west-1`, encrypted.

Dashboard now (intake flow): the call list shows caller name, phone number (formatted, e.g. (403) 555-0111), reason for the call, summary and the time of the call. The reason is labelled "Appointment request: …", "Health question: …" or "Other request: …". Clicking a call opens the full detail page. The old list columns (outcome, duration, status, assignee) are kept as a comment in the code.

Dashboard built: staff login with session cookies; call list newest first with New / Urgent / Today filters and search; call detail with summary, transcript and requested appointment; mark handled, add a note, assign;

live updates over WebSocket with a sound. **For production:** HTTPS, optional two-factor sign-in, and alerts by email or SMS ("New urgent call, check the dashboard", no patient details).

13. Connecting the clinic's existing phone number (unchanged from v1)

Option	How it works	Extra cost	Setup time	Best when
1. Call forwarding	The clinic's provider forwards calls to a hidden Twilio number connected to LiveKit	~$1–2/month + per-minute	Same day	Landline or any provider; quickest start
2. Port the number	The number moves to Twilio and goes straight into LiveKit over SIP	Usually free or a few dollars	1–3 weeks in Canada	Full control long term
3. Direct SIP	The clinic's VoIP (RingCentral, Telus Business Connect) sends calls over SIP to LiveKit	None for a number	1–3 days	Clinic already uses SIP-capable VoIP

Forwarding modes: **always**, **busy / no answer** (e.g. after 4 rings), or **after hours**. The agent reads the caller's original number from the SIP `From` header so the dashboard shows the patient's real number.

Questions for the clinic: who is the phone provider; landline or VoIP; should the AI take all calls, missed calls or after-hours calls; which number urgent calls transfer to and who answers it; who has admin access to the phone account; which booking software they use; their FAQ answers in their own words; what must always reach a human; whether they accept US vendors processing patient audio, and who signs off.

14. Timeline (rough)

When	Work	Status
7–11 Nov 2026	Phase 0 accounts; Phase 1 local system: agent, tools, guardrails, backend, database, dashboard, testing, latency	**Done**
11 Nov 2026	Client change: intake-only call flow; dashboard list cut to name, number, reason, summary	**Done, tested by voice**
Next ~1 week	Phase 2: Twilio number, SIP trunk, real phone calls, real-call latency, transfer	Needs Twilio account
When the clinic answers	Phase 3: booking system integration	Waiting on clinic
~1–2 weeks, overlapping	Phase 4: dead-air fallback, rate limiting, encryption, retention; DPAs, PIA, privacy sign-off	Paperwork depends on the clinic
~1 week, then 3 weeks of rollout	Phase 5: AWS `ca-west-1`, monitoring, backups; after-hours → overflow → all calls	After Phase 4 sign-off

15. Checklist before go-live

☐ AI disclosure and recording notice in the greeting, heard on a real phone call
☐ Emergency handling (911) tested on live phone calls, not just the Playground
☐ Human transfer tested to the clinic's real urgent number
☐ No medical advice, verified across the scripted scenario set
☐ Patient data stored in Canada and encrypted at rest
☐ Agreements signed with the clinic and every vendor (DPAs, zero retention on)
☐ Clinic's Privacy Impact Assessment complete and signed off by their privacy officer
☐ Median latency under 1.5 s on real phone calls
☐ Call logs, dashboard and staff notifications working
☐ Monitoring and alerts live, with someone on the other end of them
☐ Fallback tested: agent killed mid-call and TTS failing, calls still reach the clinic
☐ Front desk trained on the dashboard
☐ Front desk can call every caller back within 30 minutes during the hours the AI answers; after-hours closing line in place
☐ Rollback plan written: how to turn the AI off in under five minutes